Shopping Application — Speaker Notes

Slide 1 — Shopping Application

- Shopping helps an associate understand the customer, discover the right product, and gather the information needed to make a confident product decision to buy

Slide 2 — Why Shopping Matters

- Shopping is the discovery and information layer of the associate experience. The associate may start with a customer, a product, or a location question, and Shopping brings those pieces together.
- The first responsibility is customer context: identify the customer and retrieve or manage relevant customer/address information.
- The second is product discovery: search for the right item and enrich the choice with availability or installation-related context.
- The third is confidence: before the associate proceeds further in the selling experience, the product/customer/location information should be accurate enough to avoid rework or a poor customer experience.
- From an SRE perspective, this is why Shopping availability matters: a technically healthy pod is not enough if the associate cannot search, find a customer, or validate an address.

Slide 3 — Where Shopping Sits

- Explain this as a logical view, not an exact deployment diagram. The associate uses a store terminal, Hub/remote desktop, or handheld experience. The UI invokes orchestration/BFF capabilities, which then call store-shopping-service.
- store-shopping-service owns discovery and information-oriented capabilities such as item search, customer lookup, address operations, recommendations, wallet lookup, and eligibility checks.
- The Shopping service also depends on enterprise/domain systems. Therefore, when a Shopping CUJ fails, the root cause can be inside Shopping or in a downstream provider such as Search Integration, Enterprise Customer, Enterprise Address, Customer Promise, EIH, or authentication context.
- Operational takeaway: do not stop troubleshooting at the Shopping service boundary.

Slide 4 — The Shopping Journey

- Walk left to right. Authentication establishes the associate/store context. Customer lookup identifies who we are serving. Item search finds the product. Product context helps the associate understand availability or related information.
- Installation Components enriches applicable products with installation bundle/accessory recommendations. Wallet Lookup retrieves wallet-related customer context. Address Eligibility determines whether an address is eligible for the relevant experience.
- The supporting address journeys then manage, validate, suggest, or enrich the address/ZIP information.
- This is not intended to claim that every associate follows these steps in exactly this order. It is a learning model for understanding where each CUJ fits.

Slide 5 — Carry With vs Non-Carry With

- Clarify that Carry With / Non-Carry With is a business classification from the supplied CUJ inventory, not an HTTP or Kubernetes concept.
- For this session, treat Carry With / Foundational journeys as capabilities that carry important customer/product context through the core Shopping interaction. Examples are Item Search, Customer Lookup, Installation Components, Wallet Lookup, and Address Eligibility.
- Non-Carry With journeys are supporting operations invoked when the experience needs additional address work, such as add/edit address, validate address, suggest address, or ZIP information.
- The exact internal definition should be confirmed with the Shopping application team if they use “Carry With” more narrowly. The important SRE point is that both groups can still be business-critical and require monitoring.

Slide 6 — CUJ 1: Item Search

- Item Search is one of the most visible Shopping journeys. It allows the associate to search for products based on the customer request and retrieve matching item information.
- API pattern: /item/v1/item-search.+. The supplied inventory considers 2xx and 400 as accepted status semantics for the SLO and gives a 99% availability target.
- Documented dependencies include Search Integration and EIH. If this CUJ degrades, associates may be unable to find products even though the Shopping application itself is reachable.
- When troubleshooting, first check request volume, status-code distribution, latency, store/stage scope, and then downstream search/provider health.

Slide 7 — CUJ 2: Customer Lookup

- Customer Lookup finds the customer record that should be associated with the current Shopping interaction. This can support searching using the request fields accepted by the customer-management API.
- API: /customer-management/customers with the searchCustomerRequest parameters. The supplied inventory uses 2xx as success and a 99% availability target.
- The main documented dependency is Enterprise Customer. A degradation there can appear to the associate as a Shopping failure.
- Call out an important monitoring lesson: not every 400 should automatically be treated as an application failure. Invalid user input can produce expected 400 responses, so SLO semantics must follow the agreed business behavior rather than simply counting every non-2xx response as failure.

Slide 8 — CUJs 3–5: Enrich the Shopping Context

- Installation Components uses /product-recommendation/installationBundleAccessories. Its purpose is to return installation-related bundle/accessory recommendations for applicable products, helping the associate understand what else may be required for the job.
- Wallet Lookup uses /store/customer/wallet/{cardToken}. At a business level, it retrieves wallet-related context for the identified customer/card token so relevant wallet information can be surfaced during Shopping.
- Address Eligibility uses /address/addressEligibility. It answers whether the supplied address is eligible for the capability being evaluated. This can be important before presenting location-dependent choices.
- The source inventory does not provide confirmed SLO/status/dependency values for these three CUJs. Do not invent them during the session; say that the monitoring contract should be validated with the application team.

Slide 9 — CUJs 6–7: Manage and Validate Address

- Add/Edit Customer Address uses /customer-management/customers/addresses. It lets the associate create or update address information associated with the customer. The supplied inventory shows 2xx and 400 semantics, 99% availability, Enterprise Customer dependency, and alerting enabled.
- Validate Address uses /address/validateAddress. Its purpose is to check whether an entered address is valid/usable according to the address service. The inventory shows 2xx, 99% availability, Enterprise Address dependency, and alerting enabled.
- Explain why these are important: poor address quality can cause incorrect eligibility, sourcing, or later customer experience. Address operations may look secondary, but they protect data quality for the overall Shopping interaction.

Slide 10 — CUJs 8–9: Suggest Address and ZIP Info

- Suggest Address uses /address/suggestAddress. It helps the associate enter a standardized/likely address by returning suggestions while address information is being entered.
- ZIP Code Info uses /address/getZipInfo. It enriches a ZIP/postal-code input with location information required by the Shopping experience.
- These journeys reduce manual-entry errors and help ensure location-dependent decisions use clean data.
- Again, the supplied table does not contain confirmed success codes, SLOs, dependencies, or alerting values for these CUJs, so present their business function but label the operational contract as pending validation.

Slide 11 — Shopping Dependencies

- Use this slide to explain the failure chain. Search Integration supports Item Search. EIH appears in the reference mapping for Item Search. Enterprise Customer supports customer lookup and customer-address management. Enterprise Address supports address validation.
- Customer Promise is relevant to inventory, sourcing, and item-availability context. Keycloak / Store IAM supplies authentication context for the service.
- If an availability alert fires, confirm the Shopping symptom first, then determine whether it is isolated to one URI or common across several CUJs. Multiple affected CUJs can point toward a shared dependency or platform issue.
- The goal is to identify the failing layer, not simply restart the Shopping service.

Slide 12 — How We Monitor a Shopping CUJ

- Availability answers whether the business action is succeeding according to its agreed HTTP/business semantics. The numerator should represent accepted successful outcomes; the denominator represents meaningful total attempts.
- Latency p95 tells us what the slower end of normal user experience looks like without being dominated by a few extreme outliers. High p95 can hurt associates even when availability remains 100%.
- Throughput provides context. A percentage calculated from very few requests can be misleading, while a small percentage drop during heavy traffic can affect many associates.
- Status codes help separate client/business-validation behavior from server/downstream failures. Always interpret them per CUJ.
- Segment by stage, store, cluster, and URI to understand blast radius. Then correlate with provider dashboards, logs, traces, CPU/memory/JVM/pods only after confirming the business symptom.

Slide 13 — Troubleshooting Walkthrough

- Start from the CUJ alert rather than from infrastructure. First confirm that availability actually dropped and for how long.
- Check throughput next: is there enough traffic for the percentage to be meaningful? Then inspect the status-code mix to understand whether the change is mainly 4xx, 5xx, or another category.
- Narrow to the URI and then the store/stage/cluster scope. This tells you whether the issue is one API, one store, one stage, or widespread.
- Check p95 latency to see whether the journey is also slow. Then inspect Shopping infrastructure, logs and traces, and finally correlate with downstream provider health.
- The end goal is a defensible root cause: Shopping code, platform/infrastructure, bad/expected request behavior, or downstream dependency.

Slide 14 — Takeaways

- Recap four points. First, Shopping is about discovery and context: customer, product, location, recommendations and related information.
- Second, CUJs are business outcomes. We monitor Item Search because an associate must find a product—not because the URI itself is inherently important.
- Third, success semantics are API-specific. Use the agreed business definition instead of assuming every non-2xx is failure.
- Fourth, follow dependencies. A Shopping alert identifies the affected user journey, but the root cause may be an enterprise provider.
- Close by asking whether the team wants to walk through one live Grafana CUJ panel and trace it from availability to dependency health.